

## 8.1 INTRODUCTION

Data is manipulated by using the arithmetic instructions in digital computers. Data is manipulated to produce results necessary to give solution for the computation problems. The Addition, subtraction, multiplication and division are the four basic arithmetic operations. If we want then we can derive other operations by using these four operations. To execute arithmetic operations there is a separate section called arithmetic processing unit in central processing unit. The arithmetic instructions are performed generally on binary or decimal data. Fixed-point numbers are used to represent integers or fractions. We can have signed or unsigned negative numbers. Fixed-point addition is the simplest arithmetic operation. If we want to solve a problem then we use a sequence of well-defined steps. These steps are collectively called algorithm. To solve various problems we give algorithms. In order to solve the computational problems, arithmetic instructions are used in digital computers that manipulate data. These instructions perform arithmetic calculations. And these instructions perform a great activity in processing data in a digital computer. As we already stated that with the four basic arithmetic operations addition, subtraction, multiplication and division, it is possible to derive other arithmetic operations and solve scientific problems by means of numerical analysis methods. A processor has an arithmetic processor (as a sub part of it) that executes arithmetic operations. The data type, assumed to reside in processor, registers during the execution of an arithmetic instruction. Negative numbers may be in a signed magnitude or signed complement representation. There are three ways of representing negative fixed point - binary numbers signed magnitude, signed 1's complement or signed 2's complement. Most computers use the signed magnitude representation for the mantissa.

## 8.2 ADDITION AND SUBTRACTION

### Addition and Subtraction with Signed -Magnitude Data

We designate the magnitude of the two numbers by A and B. Where the signed numbers are added or subtracted, we find that there are eight different conditions to consider, depending on the sign of the numbers and the operation performed. These conditions are listed in the first column of Table 4.1. The other columns in the table show the actual operation to be performed with the magnitude of the numbers. The last column is needed to present a negative zero. In other words, when two equal numbers are subtracted, the result should be +0 not -0.

The algorithms for addition and subtraction are derived from the table and can be stated as follows (the words parentheses should be used for the subtraction algorithm)

### Signed Magnitude Addition and Subtraction

Table 8.1: Addition and subtraction of signed magnitude numbers

| Operation     | Add<br>Magnitudes | Subtract Magnitudes |              |              |
|---------------|-------------------|---------------------|--------------|--------------|
|               |                   | When $A > B$        | When $A < B$ | When $A = B$ |
| $(+A) + (+B)$ | $+(A + B)$        |                     |              |              |
| $(+A) + (-B)$ |                   | $+(A - B)$          | $-(B - A)$   | $+(A - B)$   |
| $(-A) + (+B)$ |                   | $-(A - B)$          | $+(B - A)$   | $+(A - B)$   |
| $(-A) + (-B)$ | $-(A + B)$        |                     |              |              |
| $(+A) - (+B)$ |                   | $+(A - B)$          | $-(B - A)$   | $+(A - B)$   |
| $(+A) - (-B)$ | $+(A + B)$        |                     |              |              |
| $(-A) - (+B)$ | $-(A + B)$        |                     |              |              |
| $(-A) - (-B)$ |                   | $-(A - B)$          | $+(B - A)$   | $+(A - B)$   |

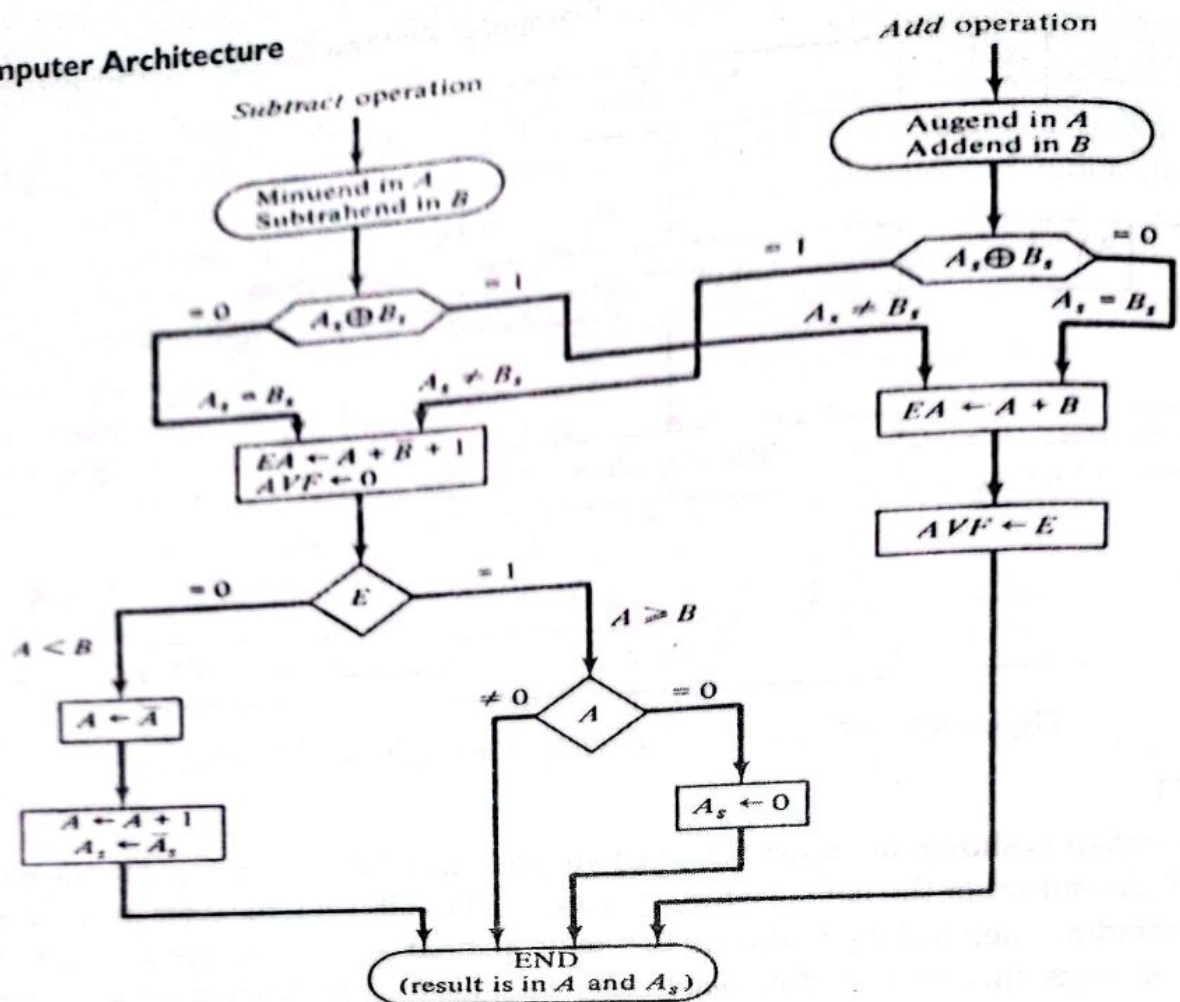


Figure 8.2: Flowchart for add and subtract operations

## Signed 2's Complement Addition and Subtraction

The algorithm for adding and subtracting two binary numbers in signed-2's complement representation is shown in the flowchart. The sum is obtained by adding the contents of AC and BR (including their sign bits). The overflow bit V is set to 1 if the exclusive-OR of the last two carries is 1, and it is cleared to 0 otherwise. The subtraction operation is accomplished by adding the content of AC to the 2's complement of BR. Taking the 2's complement of BR has the effect of changing a positive number to negative, and vice versa. An overflow must be checked during this operation because the two numbers added could have the same sign. The programmer must realize that if an overflow occurs, there will be an erroneous result in the AC register.

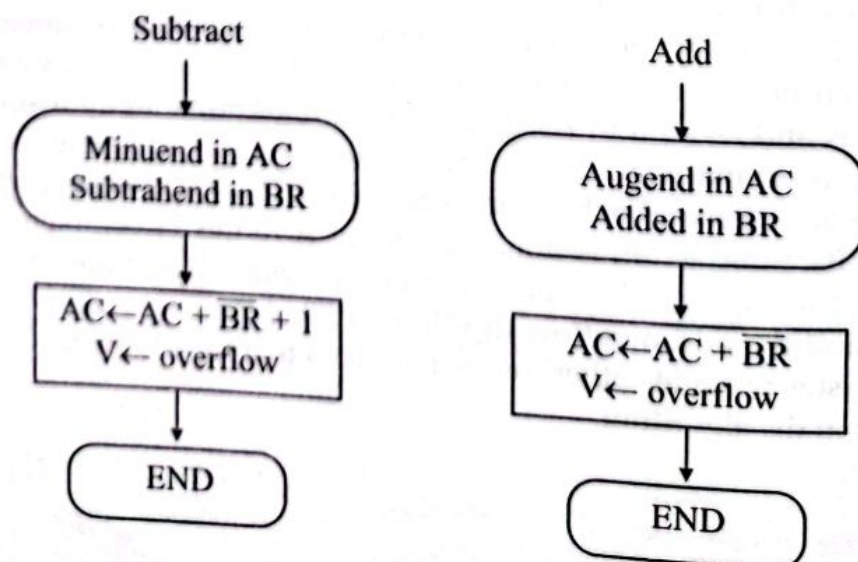


Table 8.3: Algorithm for adding and subtracting numbers in signed-2's complement representation

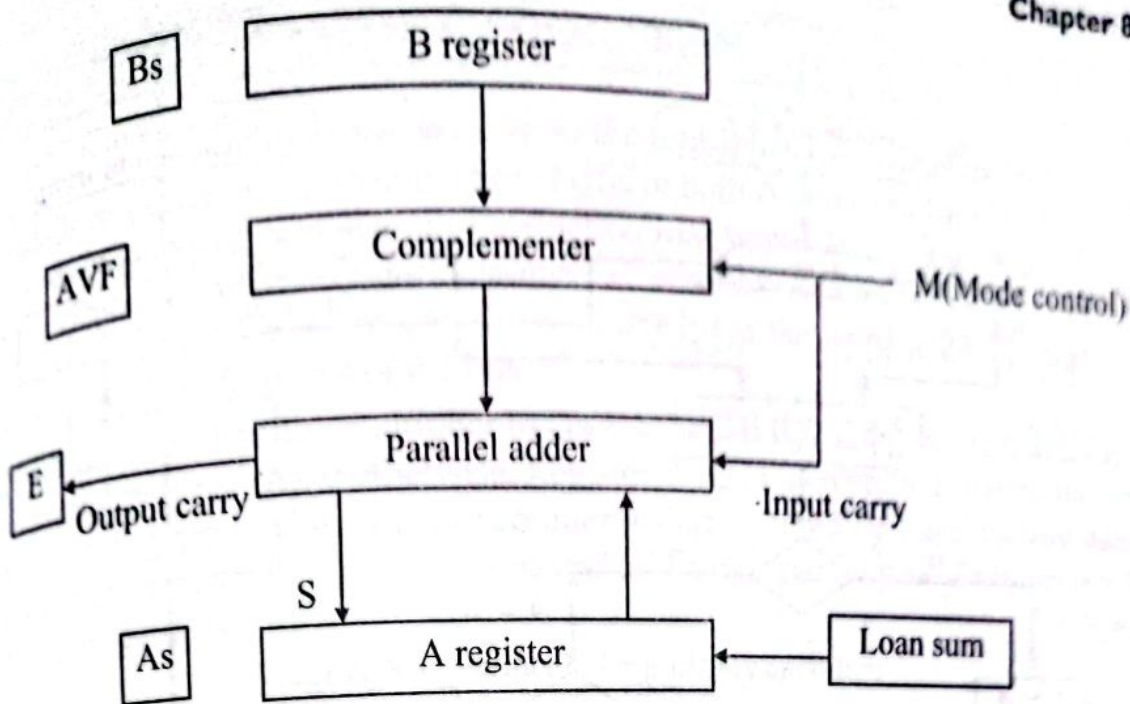


Figure 8.1: Hardware for signed magnitude addition and subtraction

gorithm

The flowchart is shown in Figure 8.2. The two signs A, and B, are compared by an exclusive-OR gate. If the output of the gate is 0 the signs are identical; if it is 1, the signs are different. For an add operation, identical signs dictate that the magnitudes be added. For a subtract operation, different signs dictate that the magnitudes be added. The magnitudes are added with a microoperation  $EA \leftarrow A + B$ , where EA is a register that combines E and A. The carry in E after the addition constitutes an overflow if it is equal to 1. The value of E is transferred into the add-overflow flip-flop AVF. The two magnitudes are subtracted if the signs are different for an add operation or identical for a subtract operation. The magnitudes are subtracted by adding A to the 2's complemented B. No overflow can occur if the numbers are subtracted so AVF is cleared to 0. The result 1 in E indicates that  $A \geq B$  and the number in A is the correct result. If this number is zero, the sign A must be made positive to avoid a negative zero and 0 in E indicates that  $A < B$ . For this case it is necessary to take the 2's complement of the value in A. The operation can be done with one microoperation  $A \leftarrow A' + 1$ . However, we assume that the A register has circuits for microoperations complement and increment, so the 2's complement is obtained from these two microoperations.

In other paths of the flowchart, the sign of the result is the same as the sign of A, so no change in A is required. However, when  $A < B$ , the sign of the result is the complement of the original sign of A. It is then necessary to complement A, to obtain the correct sign. The final result is found in register A and its sign in As. The value in AVF provides an overflow indication. The final value of E is immaterial. Figure 8.1 shows a block diagram of the hardware for implementing the addition and subtraction operations. It consists of registers A and B and sign flip-flops As and Bs. Subtraction is done by adding A to the 2's complement of B. The output carry is transferred to flip-flop E, where it can be checked to determine the relative magnitudes of two numbers and the add-overflow flip-flop AVF holds the overflow bit when A and B are added. The A register provides other microoperations that may be needed when we specify the sequence of steps in the algorithm.

### 8.3 MULTIPLICATION ALGORITHM

In the beginning, the multiplicand is in B and the multiplier in Q. Their corresponding signs are in  $B_s$  and  $Q_s$  respectively. We compare the signs of both A and Q and set to corresponding sign of the product since a double-length product will be stored in registers A and Q. Registers A and E are cleared and the sequence counter SC is set to the number of bits of the multiplier. Since an operand must be stored with its sign, one bit of the word will be occupied by the sign and the magnitude will consist of  $n-1$  bits.

Now, the low order bit of the multiplier in  $Q_n$  is tested. If it is 1, the multiplicand (B) is added to present partial product (A), 0 otherwise. Register EAQ is then shifted once to the right to form the new partial product. The sequence counter is decremented by 1 and its new value checked. If it is not equal to zero, the process is repeated and a new partial product is formed. When  $SC = 0$  we stop the process.

Figure 8.4: Hardware for multiply operation

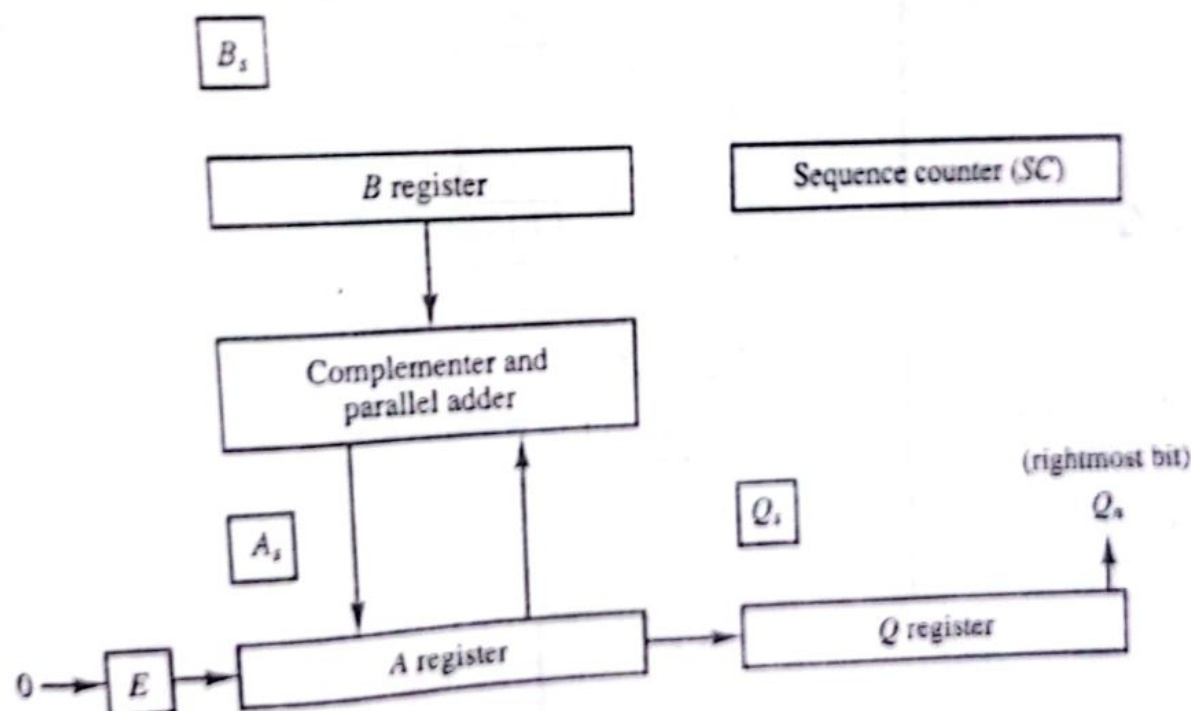


Figure 8.5: Flowchart for multiply operation

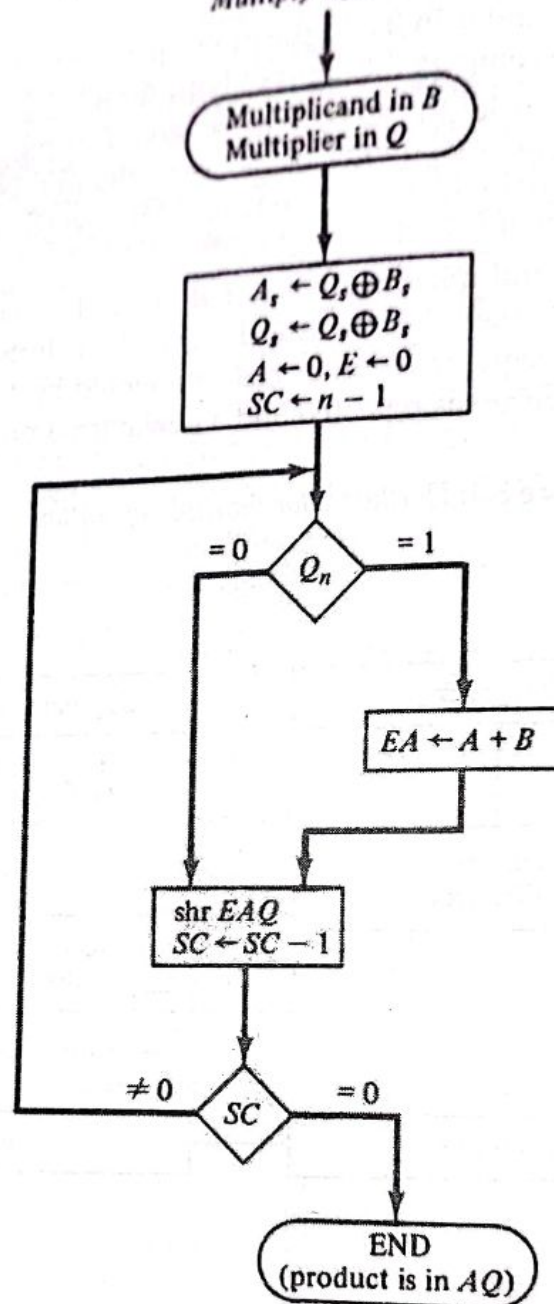


Table 8.2: Numerical example for binary multiplier

|                                  | E | A     | Q     | SC  |
|----------------------------------|---|-------|-------|-----|
| Multiplicand B = 10111           |   |       |       |     |
| Multiplier in Q                  | 0 | 00000 | 10011 | 101 |
| $Q_n = 1$ ; add B                |   | 10111 |       |     |
| First partial product            | 0 | 10111 |       |     |
| Shift right EAQ                  | 0 | 01011 | 11001 | 100 |
| $Q_n = 1$ ; add B                |   | 10111 |       |     |
| Second partial product           | 1 | 00010 |       |     |
| Shift right EAQ                  | 0 | 10001 | 01100 | 011 |
| $Q_n = 0$ ; shift right EAQ      | 0 | 01000 | 10110 | 010 |
| $Q_n = 0$ ; shift right EAQ      | 0 | 00100 | 01011 | 001 |
| $Q_n = 1$ ; add B                |   | 1011  |       |     |
| Fifth partial product            | 0 | 11011 |       |     |
| Shift right EAQ                  | 0 | 01101 | 10101 | 000 |
| Final product in AQ = 0110110101 |   |       |       |     |

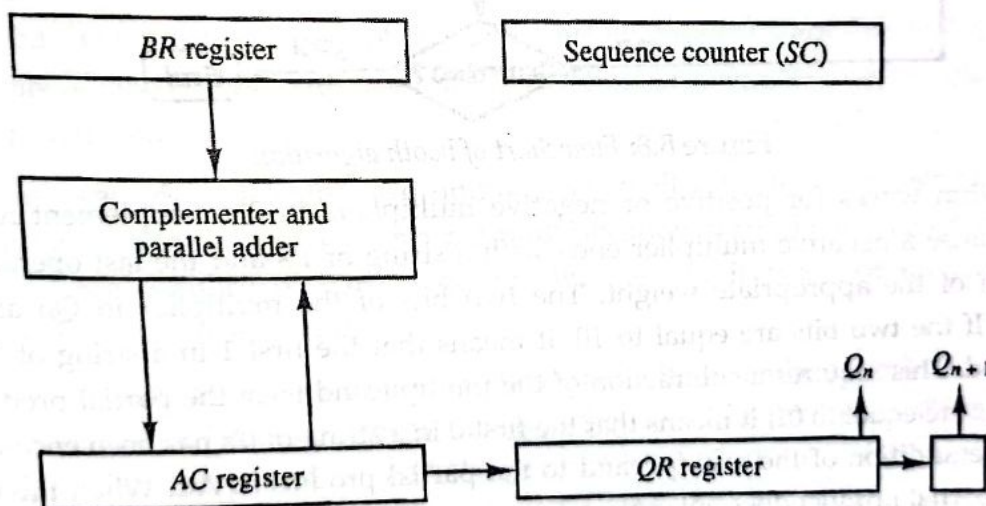
### Booth's algorithm

Booth algorithm gives a procedure for multiplying binary integers in signed-2's complement representation. It operates on the fact that strings of 0's in the multiplier require no addition but just shifting, and a string of 1's in the multiplier from bit weight  $2^k$  to weight  $2^m$  can be treated as  $2^{k+1} - 2^m$ . For example, the binary number 001110 (+14) has a string 1's from  $2^3$  to  $2^1$  ( $k=3, m=1$ ). The number can be represented as  $2^{k+1} - 2^m = 2^4 - 2^1 = 16 - 2 = 14$ . Therefore, the multiplication  $M \times 14$ , where  $M$  is the multiplicand and 14 the multiplier, can be done as  $M \times 2^4 - M \times 2^1$ . Thus the product can be obtained by shifting the binary multiplicand  $M$  four times to the left and subtracting  $M$  shifted left once.

### Hardware for Booth Algorithm

In this algorithm, sign bit are not separated from the rest of the registers. Here, registers A, B and Q are renamed as AC, BR and QR respectively. Also,  $Q_n$  designates the least significant bit of the multiplier in the register QR and flip-flop  $Q_{n+1}$  is appended to QR to facilitate a doubled bit inspection of the multiplier.

Figure 8.7: Hardware for booth algorithm



As in all multiplication schemes, booth algorithm requires examination of the multiplier bits and shifting of partial product. Prior to the shifting, the multiplicand may be added to the partial product, subtracted from the partial, or left unchanged according to the following rules:

- ▲ The multiplicand is subtracted from the partial product upon encountering the first least significant 1 in a string of 1's in the multiplier.
- ▲ The multiplicand is added to the partial product upon encountering the first 0 in a string of 0's in the multiplier.
- ▲ The partial product does not change when multiplier bit is identical to the previous multiplier bit.

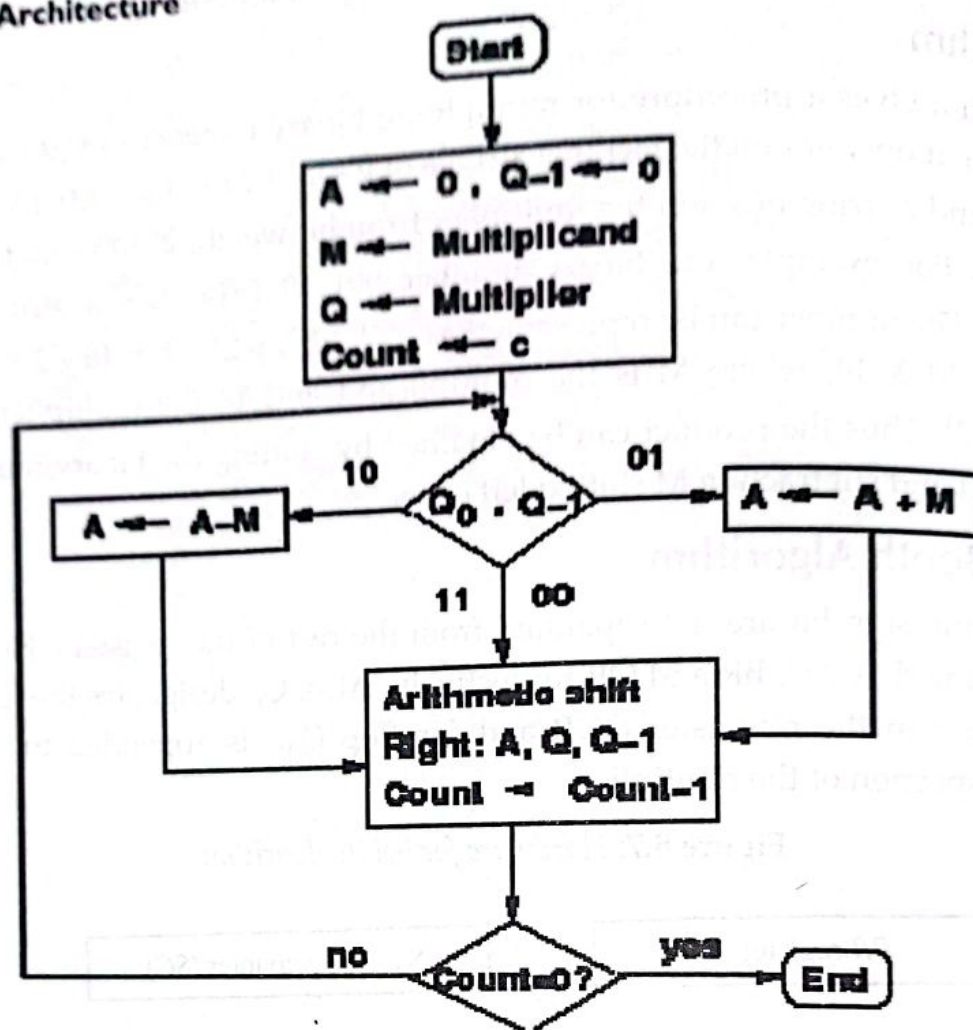


Figure 8.8: Flowchart of booth algorithm

The algorithm works for positive or negative multipliers in 2's complement representation. This is because a negative multiplier ends with a string of 1's and the last operation will be a subtraction of the appropriate weight. The two bits of the multiplier in  $Q_n$  and  $Q_{n+1}$  are inspected. If the two bits are equal to 10, it means that the first 1 in a string of 1's has been encountered. This requires a subtraction of the multiplicand from the partial product in AC. If the two bits are equal to 01, it means that the first 0 in a string of 0's has been encountered. This requires the addition of the multiplicand to the partial product in AC. When the two bits are equal, the partial product does not change.

A numerical example of Booth algorithm is shown in table 8.3 for  $n = 5$ . It shows the step-by-step multiplication of  $(-9) \times (-13) = +117$ . Note that the multiplier in QR is negative and that the multiplicand in BR is also negative. The 10-bit product appears in AC and QR and is positive. The final value of  $Q_{n+1}$  is the original sign bit of the multiplier and should not be taken as part of the product.

| $Q_n Q_{n+1}$ | $BR = 10111$<br>$\overline{BR} + 1 = 01001$ | AC    | QR    | $Q_{n+1}$ | SC  |
|---------------|---------------------------------------------|-------|-------|-----------|-----|
|               | Initial                                     | 00000 |       |           |     |
| 1 0           | Subtract $BR$                               | 01001 | 10011 | 0         | 101 |
|               |                                             | 01001 |       |           |     |
|               | ashr                                        | 00100 | 11001 |           |     |
| 1 1           | ashr                                        | 00010 | 01100 | 1         | 100 |
| 0 1           | Add $BR$                                    | 10111 |       | 1         | 011 |
|               |                                             | 11001 |       |           |     |
|               | ashr                                        | 11100 | 10110 | 0         | 010 |
| 0 0           | ashr                                        | 11110 | 01011 | 0         | 001 |
| 1 0           | Subtract $BR$                               | 01001 |       |           |     |
|               |                                             | 00111 |       |           |     |
|               | ashr                                        | 00011 | 10101 | 1         | 000 |

Table 8.3: Example of Multiplication with Booth Algorithm

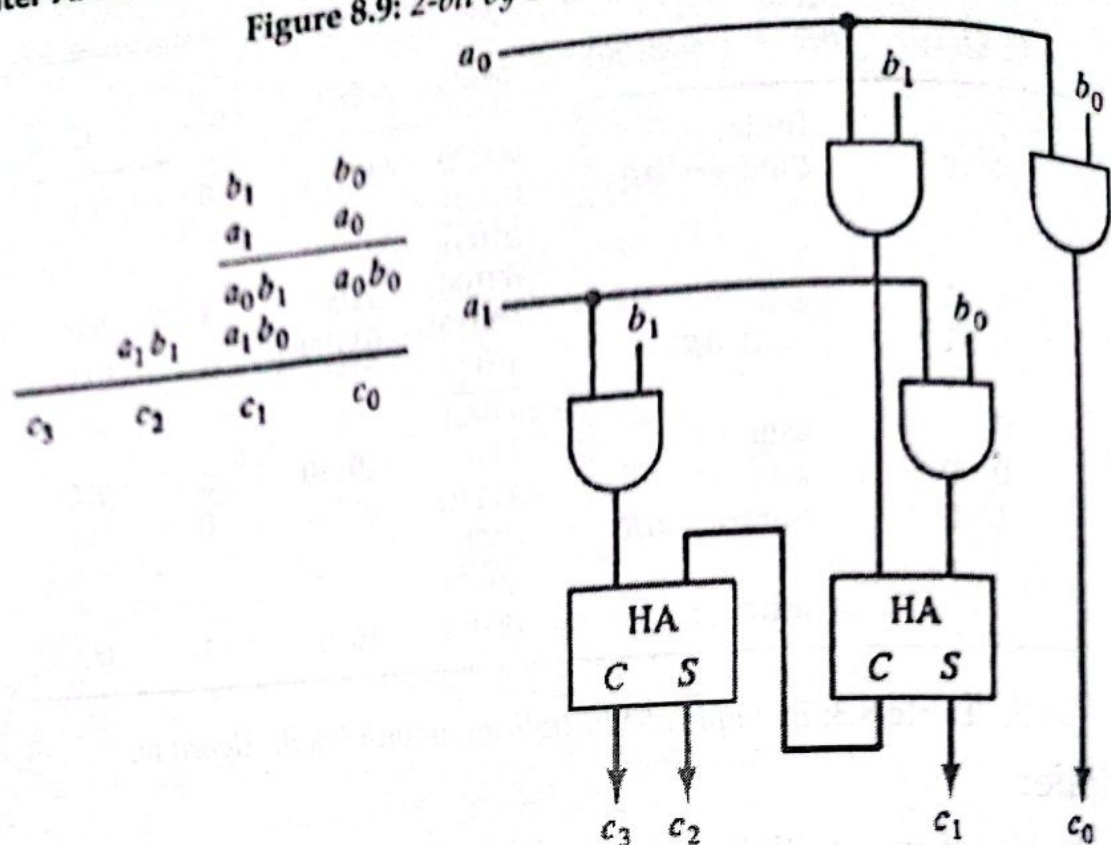
## Array Multiplier

Checking the bits of the multiplier one at a time and forming partial products is a sequential operation that requires a sequence of add and shift microoperations. The multiplication of two binary numbers can be done with one micro-operation by means of a combinational circuit that forms the product bits all at once.

This is a fast way of multiplying two numbers since all it takes is the time for the signals to propagate through the gates that form the multiplication array. However, an array multiplier requires a large number of gates, and for this reason it was not economical until the development of integrated circuits.

To see how an array multiplier can be implemented with a combinational circuit, consider the multiplication of two 2-bit numbers as shown in Fig. 10-9. The multiplicand bits are  $b_1$  and  $b_0$ , the multiplier bits are  $a_1$  and  $a_0$ , and the product is  $c_3 c_2 c_1 c_0$ . The first partial product is formed by multiplying  $a_0$  by  $b_1 b_0$ . The multiplication of two bits such as  $a_0$  and  $b_0$  produces a 1 if both bits are 1; otherwise, it produces a 0. This is identical to an AND operation and can be implemented with an AND gate. As shown in the diagram, the first partial product is formed by means of two AND gates. The second partial product is formed by multiplying  $a_1$  by  $b_1 b_0$  and is shifted one position to the left. The two partial products are added with two half-adder (HA) circuits. Usually, there are more bits in the partial products and it will be necessary to use full-adders to produce the sum. Note that the least significant bit of the product does not have to go through an adder since it is formed by the output of the first AND gate.

Figure 8.9: 2-bit by 2-bit array multiplier



A combinational circuit binary multiplier with more bits can be constructed in a similar fashion. A bit of the multiplier is ANDed with each bit of the multiplicand in as many levels as there are bits in the multiplier. The binary output in each level of AND gates is added in parallel with the partial product of the previous level to form a new partial product. The last level produces the product. For  $j$  multiplier bits and  $k$  multiplicand bits we need  $j \times k$  AND gates and  $(j - 1)k$ -bit adders to produce a product of  $j + k$  bits. As a second example, consider a multiplier circuit that multiplies a binary number of four bits with a number of three bits. Let the multiplicand be represented by  $b_3 b_2 b_1 b_0$  and the multiplier by  $a_2 a_1 a_0$ . Since  $k = 4$  and  $j = 3$ , we need 12 AND gates and two 4-bit adders to produce a product of seven bits. The logic diagram of the multiplier is shown in Figure above.

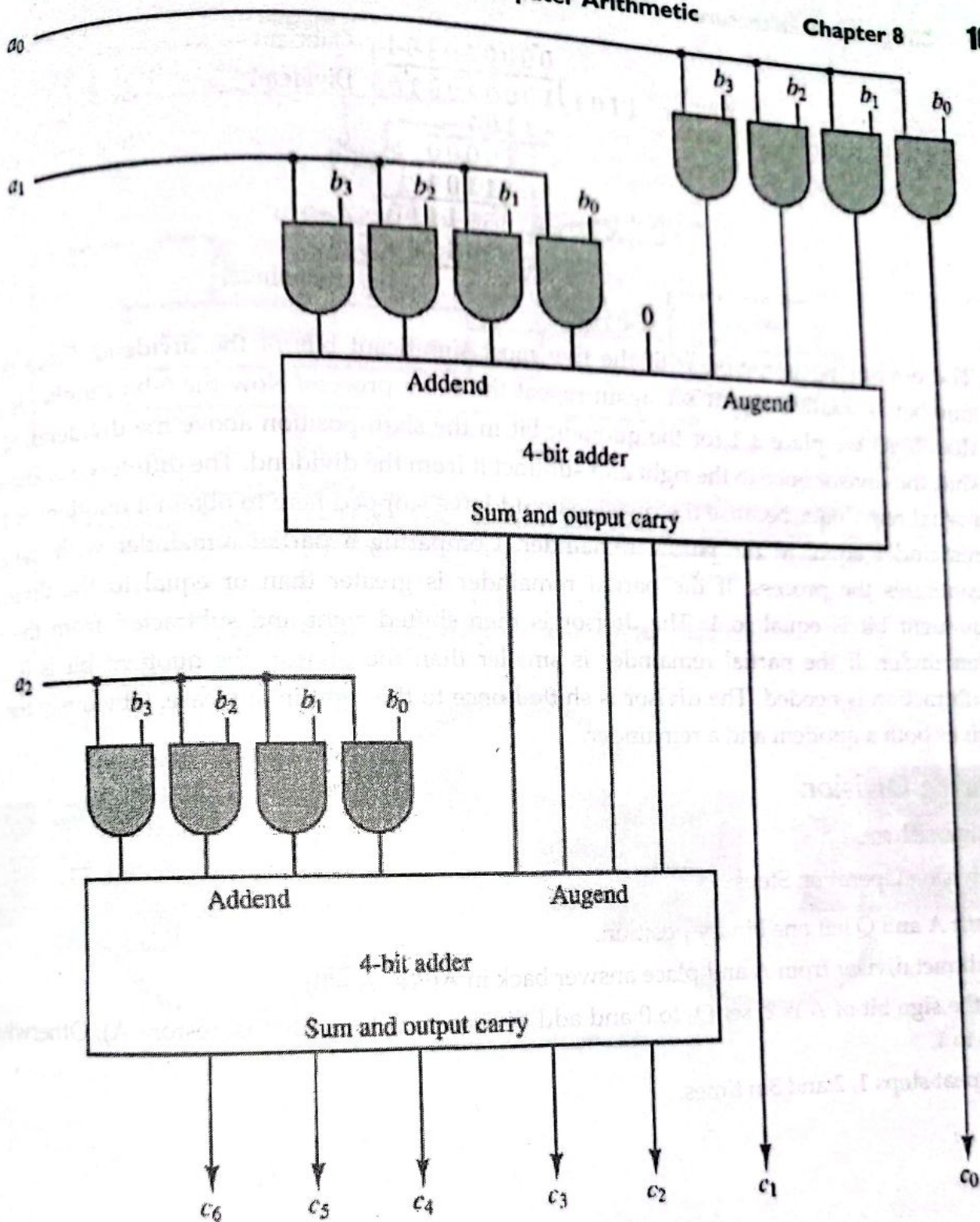


Figure 8.10: 4-bit by 3-bit array multiplier

## 8.4 DIVISION ALGORITHMS

Division of two fixed-point binary numbers in signed magnitude representation is performed with paper and pencil by a process of successive compare, shift and subtract operations. Binary division is much simpler than decimal division because here the quotient digits are either 0 or 1 and there is no need to estimate how many times the dividend or partial remainder fits into the divisor. The division process is described in Figure

|         |      |                                                                                                                                             |                                                               |
|---------|------|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Divisor | 1101 | $  \begin{array}{r}  000010101 \\  100010010 \\  -1101 \\  \hline  10000 \\  -1101 \\  \hline  1110 \\  -1101 \\  \hline  1  \end{array}  $ | Quotient<br>Dividend<br><br><br><br><br><br><br><br>Remainder |
|---------|------|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|

The divisor is compared with the five most significant bits of the dividend. Since the 5-bit number is smaller than B, we again repeat the same process. Now the 6-bit number is greater than B, so we place a 1 for the quotient bit in the sixth position above the dividend. Now we shift the divisor once to the right and subtract it from the dividend. The difference is known as a partial remainder because the division could have stopped here to obtain a quotient of 1 and a remainder equal to the partial remainder. Comparing a partial remainder with the divisor continues the process. If the partial remainder is greater than or equal to the divisor, the quotient bit is equal to 1. The divisor is then shifted right and subtracted from the partial remainder. If the partial remainder is smaller than the divisor, the quotient bit is 0 and no subtraction is needed. The divisor is shifted once to the right in any case. Obviously the result gives both a quotient and a remainder.

## Restoring Division

### Algorithm

Division Operation Steps:

1. Shift A and Q left one binary position.
2. Subtract divisor from A and place answer back in A ( $A \leftarrow A - B$ ).
3. If the sign bit of A is 1, set  $Q_0$  to 0 and add divisor back to A (that is, restore A); Otherwise, set  $Q_0$  to 1.
4. Repeat steps 1, 2 and 3 n times.

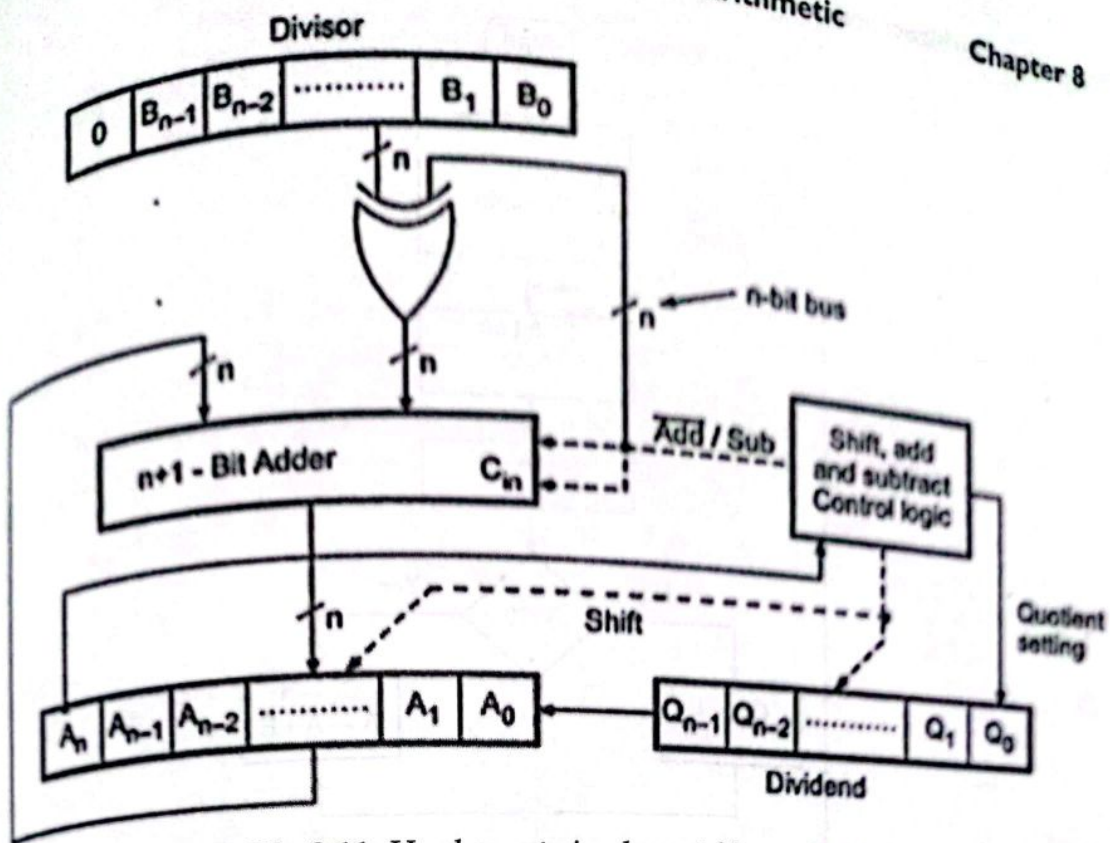


Table 8.11: Hardware to implement binary division

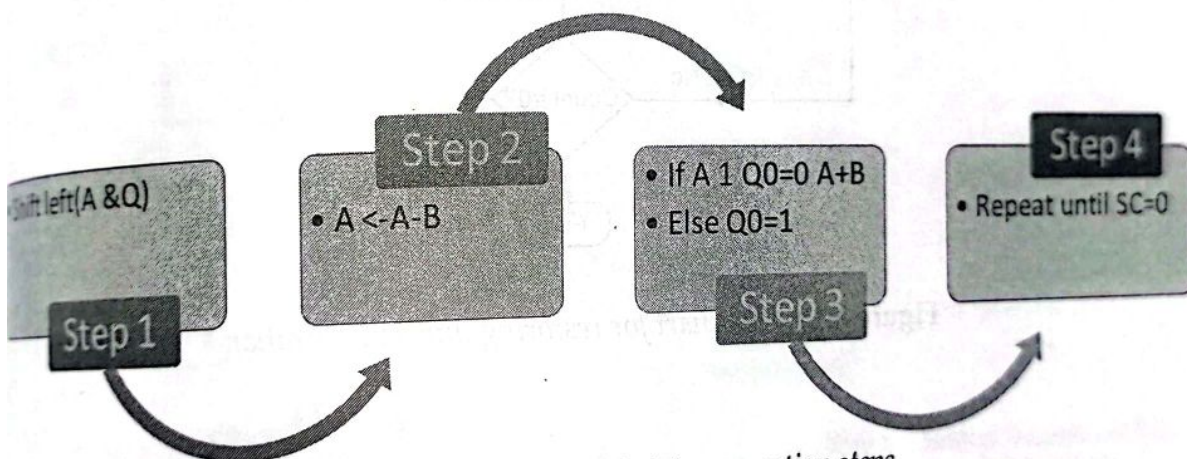


Figure 8.12: Restoring division operation steps

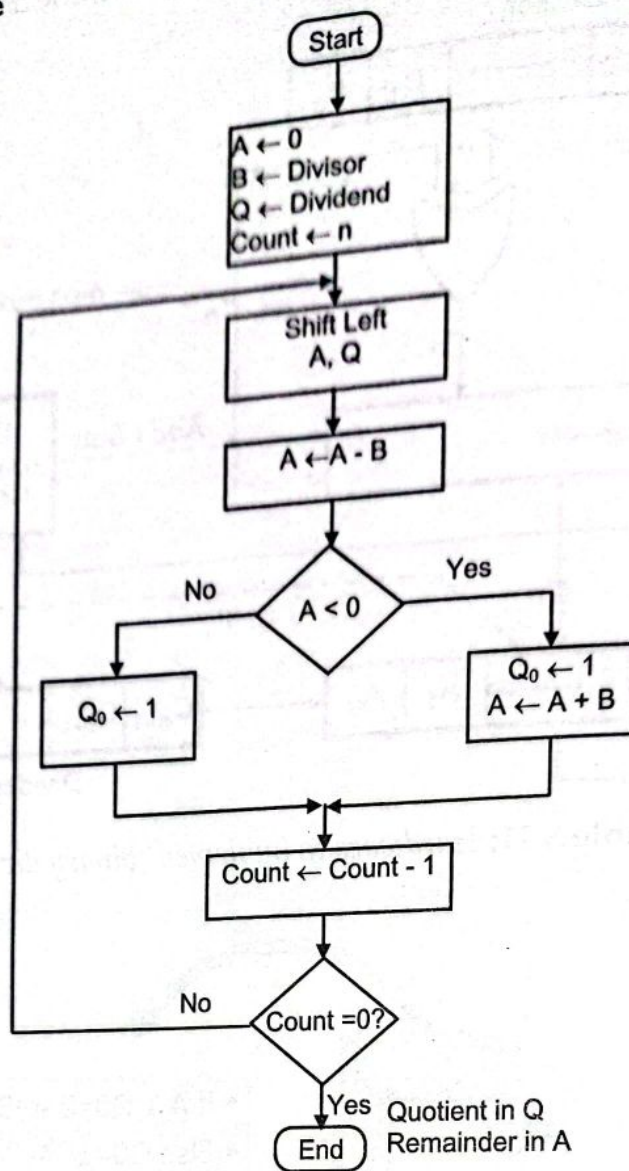


Figure 8.13: Flowchart for restoring division operation

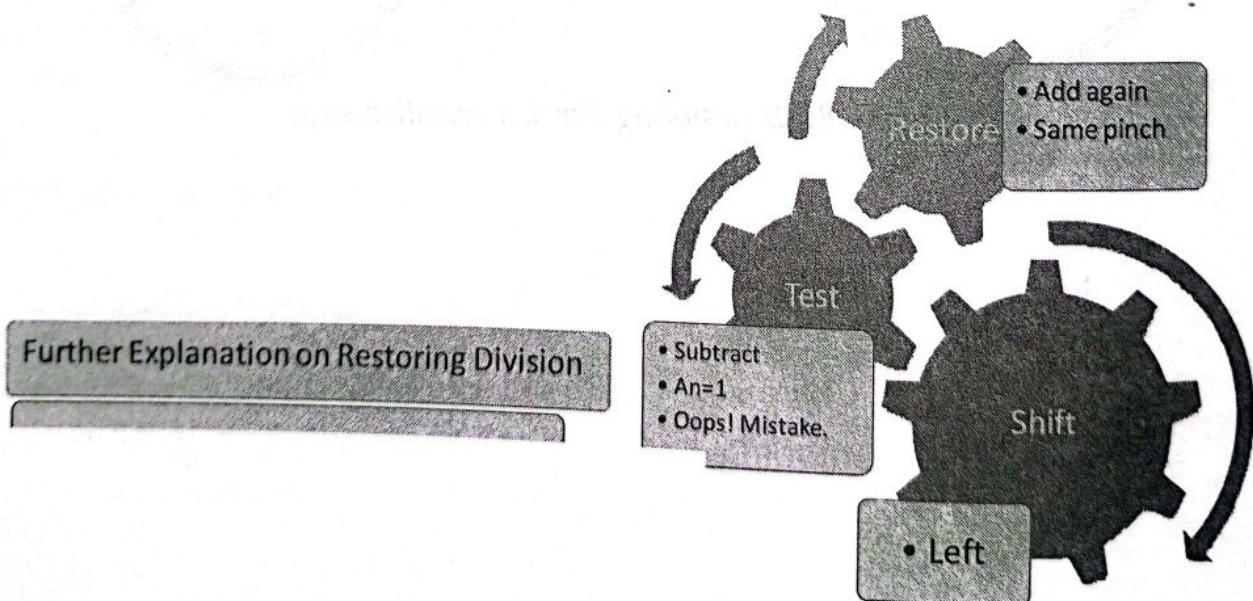
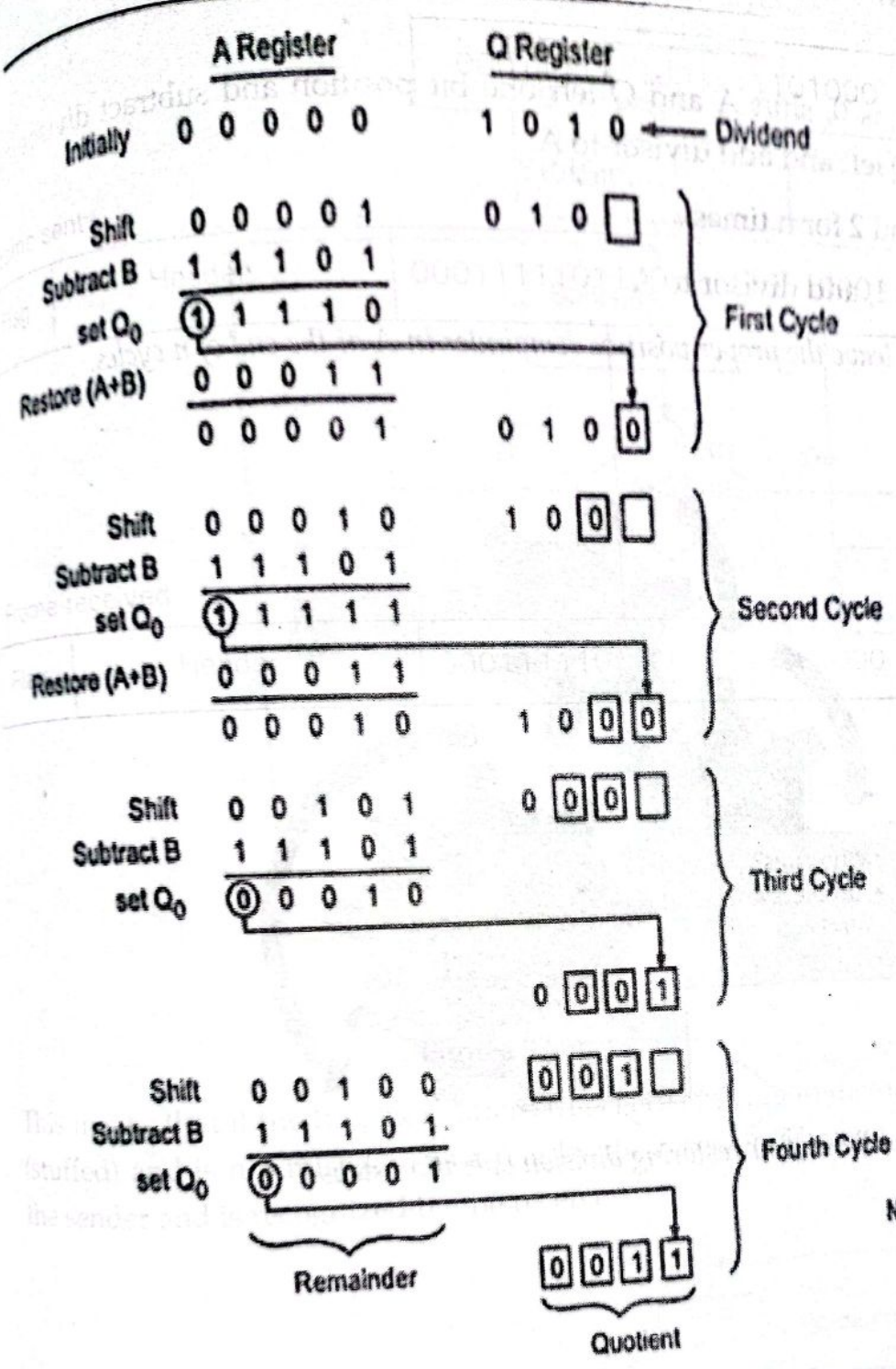


Figure 8.14: Further explanation on restoring division

Table 8.4: Restoring division example



Note : Subtract B means add B in 2's complement form

Figure: 8.15: A restoring division example

|                                                      |                                                                     |
|------------------------------------------------------|---------------------------------------------------------------------|
| If A is positive                                     | If A is negative                                                    |
| Shift left and subtract divisor $\rightarrow 2A - B$ | Restore $\rightarrow A + B$                                         |
|                                                      | Shift left and subtract divisor $\rightarrow 2(A + B) - B = 2A + B$ |

## Non-Restoring Division

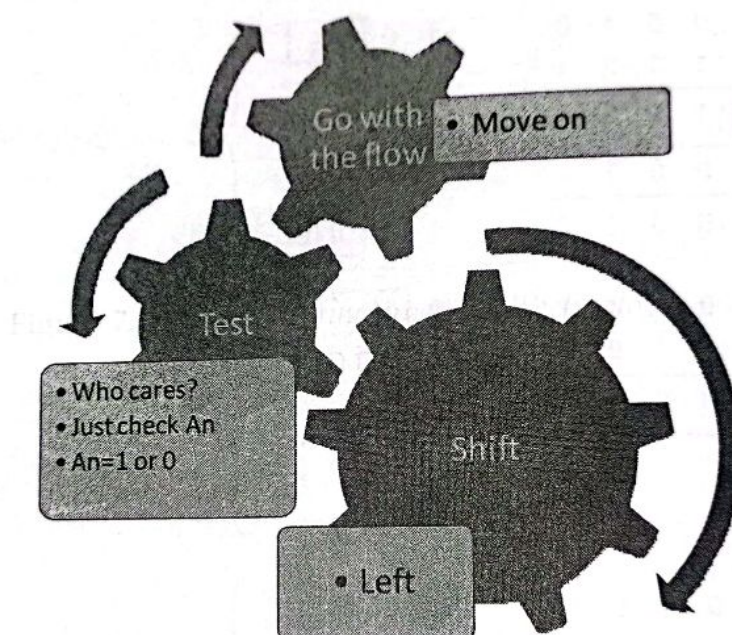
### Algorithm

**Step 1:** If the sign of A is 0, shift A and Q left one bit position and subtract divisor from A; otherwise, shift A and Q left and add divisor to A.

**Step 2:** Repeat steps 1 and 2 for n times.

**Step 3:** If the sign of A is 1, add divisor to A.

**Note:** Step 3 is required to leave the proper positive remainder in A at the end of n cycles.



**Figure 8.16:** Non-restoring division operation steps

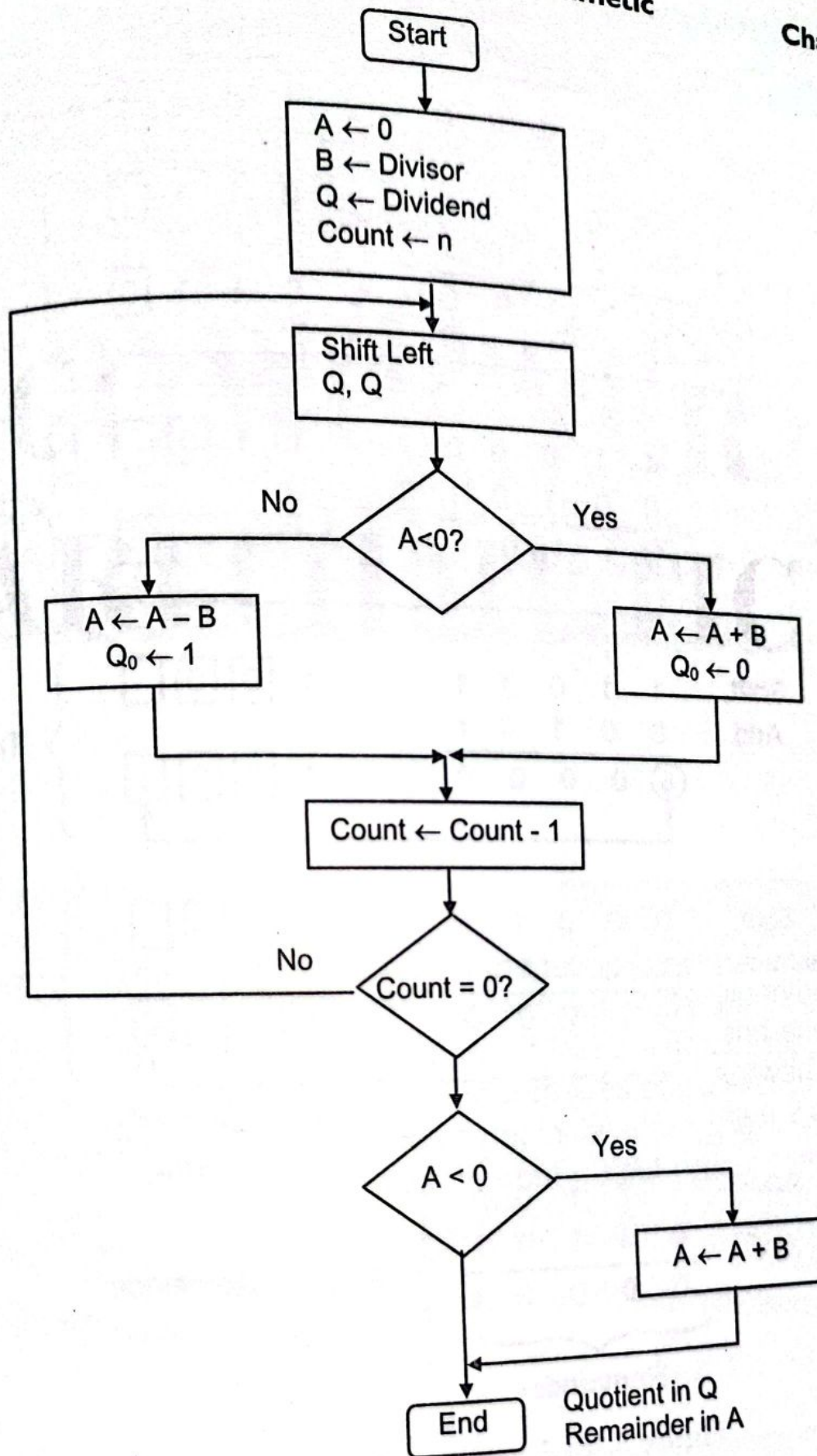


Figure 8.17: Flowchart for non-restoring division operation

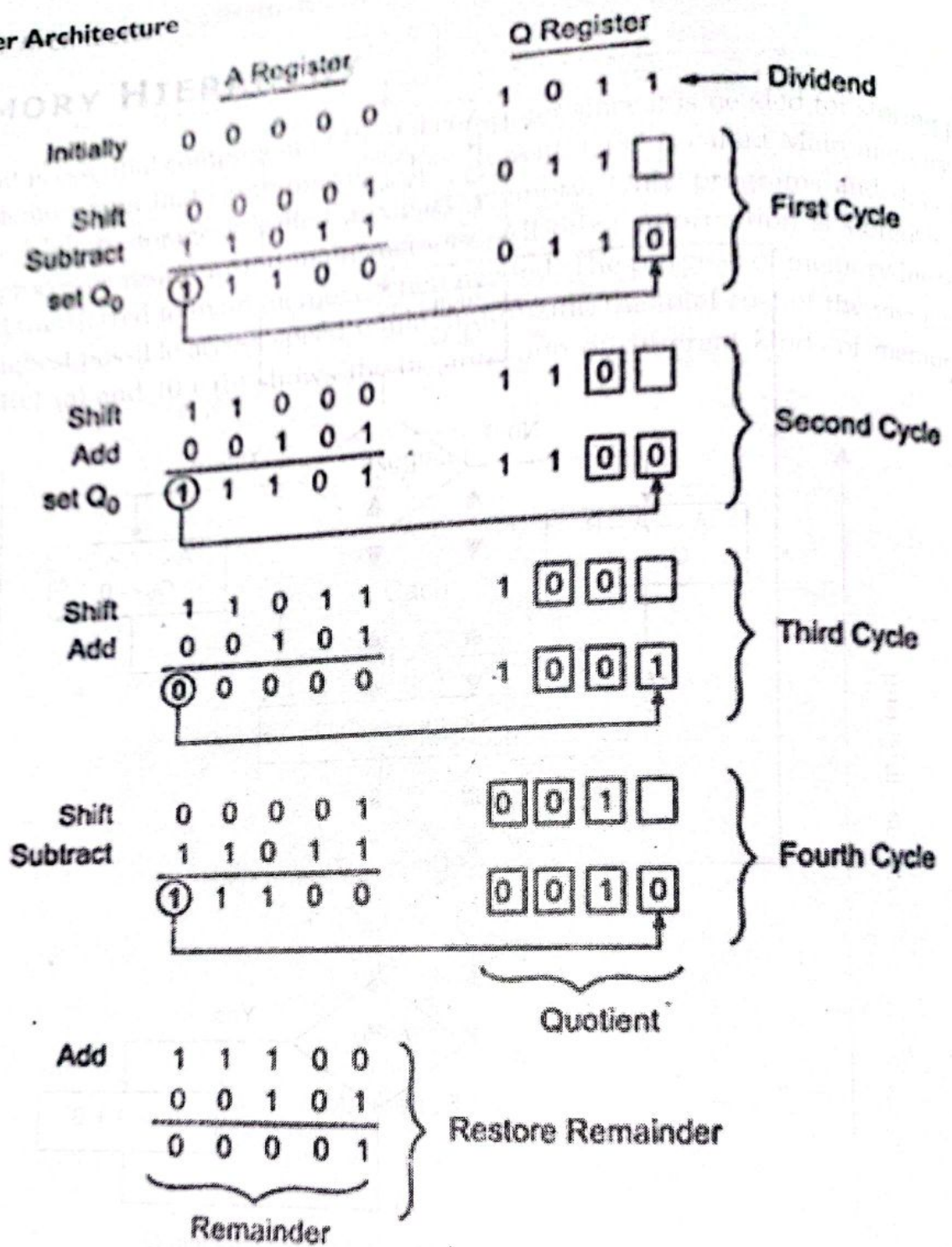


Figure 8.18: A non-restoring division example

